Quest Pro enhanced face tracking

V3.0.0 release candidate | USB or wireless ADB | 11 October 2026

A rooted Meta Quest Pro and the latest VRCFaceTracking from Steam are required. If you used a version before V2.0, record and train a new tongue model. Working V2.0 or newer models can be exported and imported into V3.0.0. Developer v8 remains the default. Mustachio is optional and highly experimental.

Based on Qpro-Enhanced-FT by n0tmast3r. Fwooffy maintains the AMD/NVIDIA wireless edition. Fwooffy wrote the original Quest Pro root guide; glorpette made and hosts the linked GitHub version.

Start with the headset's eye tracking, then set up the PC software. The **Hub** is the `QproFaceTracking.exe` program. **ADB** is the connection it uses to talk to your Quest. The **Qpro module** sends tracking results to VRCFaceTracking.

1. Set up eye tracking and independent gaze first

1. On the rooted Quest Pro, open **Settings > Hands & Eyes**. Turn on **Eye Tracking** and **Natural Facial Expressions**, then run eye calibration if offered. If these are already enabled, leave them on. Keep Developer Mode enabled.
2. If **Magisk OverlayFS** is not installed yet, open **Singularity > Apps/Modules > Magisk repo** and install it. Follow any reboot prompts. If it is already enabled in Magisk, you can skip reinstalling it.
3. In the same Singularity menu, install **Quest Pro Independent Eye Gaze** if it is not installed yet. Reboot if prompted, then check in Magisk that both modules are enabled. This is the headset-side independent gaze route.
4. Once your selected headset streaming app and VRCFaceTracking are installed in section 3, check that left and right gaze and convergence move correctly. The Magisk route has been reported working on **Horizon OS v2.7**, but exact firmware builds can behave differently.

Already using the Independent Eye Gaze Magisk module? If it is installed and enabled, it already provides independent gaze. **Skip Check gaze setup and Prepare gaze, and leave Independent Eye Gaze off in the Hub.** Neither button is required to use the Magisk method. You can still use Qpro's tongue, camera cheek, pupil dilation and eyebrow features.

Use one independent gaze method at a time. While the Magisk module is active, leave **Independent Eye Gaze** unchecked in the Hub. If you want to try the Hub's temporary gaze method instead, follow **Check gaze setup** and **Prepare gaze** in section 5 before turning on **Independent Eye Gaze**. Applying that method briefly freezes the headset while its tracking service restarts; wait for Activity to confirm it is ready. **Stop tracking** only reverses the Hub's changes; it does not disable a Magisk module.

Before switching from the Magisk gaze method to the Hub's method, disable the gaze module in Magisk and reboot the headset. The same step is needed if you want ordinary headset eye tracking again. The Hub leaves Magisk modules alone.

2. Download and open the Hub

1. Download **QproFaceTracking.V3.0.0.zip** when it is published on this project's Releases page, or use the supplied release candidate ZIP. Choose the named QproFaceTracking ZIP, **not** GitHub's "Source code" ZIP.
2. In Windows File Explorer, right-click the ZIP and choose **Extract All**. Open the extracted folder. Keep `QproFaceTracking.exe`, `Helpers`, and `QproRuntime` together.
3. Double-click `QproFaceTracking.exe`. It opens on **First-time setup** the first time. You can return to that page from the menu on the left.

You need an internet connection and several gigabytes of free space for setup. The ZIP already includes ADB. The Hub installs the Python parts it needs, so you do not have to install ADB or Python separately for this program.

Keep the Hub updated

The Hub checks GitHub for a newer stable release when it opens. Click **Check updates** or **Update available** in the top-right corner to read the release notes. You can turn off **Check automatically when the Hub opens** in that window and check manually whenever you want.

Enable **Include prereleases** in the update window to check published test releases too. This is off by default and your choice is remembered. Updates still require clicking **Update** and **Restart and update**. A release must have a newer three-part version number, such as `v3.0.1`; a different test ZIP with the same version is not offered as an update. Suffix tags such as `v3.0.1-beta.1` require a manual download.

1. Stop tracking, then click **Update** to download the release and verify its ZIP checksum and packaged files. You can cancel while the download is running; the current app stays unchanged.
2. After verification, click **Restart and update**. The Hub closes, replaces its app files in the same folder, then reopens. It keeps your models, recordings, enabled options and connection settings. Press **Start tracking** when you are ready; tracking does not start automatically after the restart.
3. Open **Components** in the update window. It checks the installed Qpro module and Qpro's private Python, PyTorch and ROCm environments against the new release. Click **Update components** to apply the listed changes. Close VRCFaceTracking and wait for its module process to exit before updating its module; stop Qpro tracking and training before updating any component. Reopen VRCFaceTracking after its module changes.

Component updates preserve the installed module's streaming app and the current or legacy ROCm track. They do not install optional components you have never enabled or change another application's Python. Older installations without a completed recipe record are offered a one-time verify/update. Custom runtime overrides are left for manual maintenance in **First-time setup**. The component window shows download and validation output; an error stops the remaining updates and explains what needs attention.

PC runtime changes retain a recovery copy and restore it if installation or verification fails. An abruptly terminated process can leave that recovery copy for manual repair. ROCm updates use a separate environment and retain the prior verified environment until GPU training and inference checks pass. Several gigabytes of extra free space may be needed during updates.

The updater stages files in a temporary folder and keeps a backup for rollback if replacement fails. If a bundled model collides with your existing model, it keeps your entire existing model pair and reports that the new bundled model was skipped. If GitHub provides no ZIP checksum or the package does not support the Hub updater, use **View release on GitHub** and download the release ZIP manually. Older builds without the updater can be tested from a new extracted folder; keep the original folder and import working V2.0 or newer tongue models through **Model manager**.

3. Get the PC apps ready

If Magisk asks whether **Shell / ADB Shell** may have root access, allow it. Keep the Quest awake while setting up the Hub.

Choose either Virtual Desktop and its PC Streamer or Steam Link for your Quest-to-PC VR connection. Install SteamVR and the latest VRCFaceTracking from Steam on the PC. **The current Steam VRCFaceTracking is required for either source.** Let Steam finish updating it before installing Qpro's module. After installing the matching Qpro module in section 5, try ordinary face and blink tracking before turning on Qpro's extra features.

Virtual Desktop settings

Open **Virtual Desktop on the Quest**, connect to the PC Streamer, then open the headset app's **Streaming** tab. These options are in the headset app, not the Windows Streamer window; expand **Advanced Options** if needed.

Setting	Use for the initial Qpro / VRChat setup
Forward tracking data to PC	**On.** Accept **Yes** when Virtual Desktop asks to share tracking data. Eye and face tracking must also be enabled in Quest settings, with permission granted to Virtual Desktop.
Track controllers	**On** when using your Touch Pro controllers, including Qpro's hands + controllers option. Turning this off prevents Virtual Desktop from adding the Quest controllers to SteamVR.
Emulate Index controllers	**Off** for the initial VRChat finger-tracking check. Virtual Desktop's native VRChat hand-tracking instructions use forwarding with Index emulation unchecked. Qpro routes optical fingers to the held controller's existing skeleton; Index emulation is not a Qpro requirement.

These settings follow VRCFaceTracking's Virtual Desktop setup and Virtual Desktop's own VRChat hand-tracking release notes. Menu wording can change between Virtual Desktop versions. Emulated Vive trackers are an optional body-tracking feature; they are not required for Qpro face tracking or fingers.

Start the PC Streamer and connect inside Virtual Desktop **before** opening VRCFaceTracking. Launch **SteamVR through Virtual Desktop**, then the SteamVR version of VRChat. In VRChat, turn on **Main Menu > Controls > Avatars Use Finger Tracking** if using optical fingers. In the Hub choose **Virtual Desktop** as the **Streaming app**, then install its matching Qpro module with VRCFaceTracking closed. Reopen VRCFaceTracking and check that ordinary face expressions move before enabling Qpro camera features.

Hands + controllers also requires Quest hand tracking and Singularity's simultaneous switch. The Virtual Desktop options above do not enable those for you. Follow the optional hands instructions in section 5; its compatibility check still limits the current experimental headset profile to **Virtual Desktop 1.34.22.0**. Other headset versions may work for ordinary face tracking without supporting Qpro hands.

If you want to stream the **Virtual Desktop picture over a USB cable**, Fwooffy's working setup used the Virtual Desktop **Beta** channel. In the Meta Horizon phone app or the headset's app library, long-press Virtual Desktop, open **Settings > Release Channels**, choose **Beta**, and update it. This changes VR video streaming. The Hub's USB or wireless choice below controls **tracking data**, which is separate. A newer beta can fall outside the checked hands profile; keep Hands + controllers off if its compatibility check rejects that version.

Steam Link settings

You can choose **Steam Link** under **Streaming app** instead of Virtual Desktop. SteamVR and the latest VRCFaceTracking are still required. In Steam Link on the headset, open **Advanced Settings**, turn on **OSC**, **Share eye tracking data**, and **Share face tracking data**, then set **OSC Output Port** to **9015**. These are Steam Link's tracking settings; the Hub's **Connection type** still chooses USB or wireless ADB for Qpro's headset camera stream.

Close VRCFaceTracking and wait for its module process to exit before switching sources. In **First-time setup**, choose **Steam Link** and press **Install Steam Link module**; reopen VRCFaceTracking afterward. Installing this Qpro module removes the Qpro Virtual Desktop module. A separate LinkFT or Steam Link VRCFaceTracking module still needs to be removed through VRCFaceTracking because it competes for the same OSC port and eye/face slots. Steam Link face, blink, and mouth tracking worked after a headset restart, Qpro's camera tongue override moved in VRChat, and native Steam TongueOut worked with VRCFaceTracking running on Steam Link. To return to Virtual Desktop, choose it in the Hub, press **Install Virtual Desktop module**, then reopen VRCFaceTracking.

4. Connect your Quest to the Hub

Choose **one** connection type on **First-time setup**. If this is your first time using the Hub, USB is the easiest way to check that it works. You can switch to wireless later.

Option A: USB cable

1. Plug the Quest Pro into the PC and put the headset on.
2. If a **USB debugging** message appears in the headset, choose **Always allow from this computer**, then **Allow**. This approval comes from the headset; the Hub cannot grant it for you.
3. In the Hub, set **Connection type** to **USB cable**. Qpro remembers the USB headset after checking that it is a Quest Pro. On later starts, leave the cable connected and keep the Quest awake; you should not normally need to unplug it. If the Hub says it can see the headset but has no root access, open Magisk on the Quest and allow **Shell / ADB Shell**. Then use **Refresh connection status** on the Live tracking page.
4. If the USB connection stalls, stop Qpro tracking and training, then press **Reconnect USB** in **First-time setup**. It retries the selected USB connection and reports whether the Quest needs debugging approval, is offline, or is missing. It does not reconnect other Android devices or change to wireless automatically. A loose cable or missing USB driver still needs to be fixed separately.

To use a different Quest Pro, stop Qpro tracking and training, press **Forget USB headset**, then connect the intended headset and press **Reconnect USB** to verify and remember it. After forgetting, automatic status checks wait for that button instead of selecting the still-connected old headset again. If several USB devices are connected before a Quest Pro is remembered, disconnect the other devices for the first check. Qpro does not silently replace a remembered headset with a different one. The saved USB selection stays with this app's connection settings when the updater runs.

Option B: Wireless ADB

1. Connect the Quest and PC to the same private Wi-Fi network. On **First-time setup**, choose **Wireless ADB (Wi-Fi)**.
2. Enter the Quest's Wi-Fi address in **Quest IP:port**. It usually looks like `192.168.1.25:5555`. The numbers before `:5555` are *your Quest's* address, so do not copy that example exactly. Press **Connect to Quest**.
3. A **Quest connected** pop-up means wireless ADB and Magisk root are ready. Activity will also say **Wireless Quest ready**. **Do not press Pair and connect.** You can unplug the USB cable, if one was attached. If a **Quest connection failed** pop-up appears, check the Quest's current IP and port, keep it awake, and read **Activity** for the exact error.
4. If the wireless connection is off but USB works, plug in USB once and press **Enable from USB**. After the Hub reports success, unplug USB and connect to the saved Quest address.

Pair and connect is only for a headset that shows a **six-digit wireless debugging pairing code**. Keep the pairing message open in the headset. Press **Show pairing options** in the Hub. Put the short-lived address shown next to the code into **Pairing IP:port**, put the code into **Six-digit code**, and keep the normal Quest address in **Quest IP:port**. The two port numbers are different. For example, `192.168.1.25:5555` is a normal connection address, **not** a pairing address. Use your Quest's actual address.

If pairing says `protocol fault`, open a new pairing message on the headset and use its new code and pairing port. If **Connect to Quest** already said ready, skip pairing. Your Quest's Wi-Fi address may change after a reboot; update it in the Hub if it does. Use wireless ADB only on a trusted network.

5. Press the setup buttons

On **First-time setup**, work down the three numbered cards:

1. **Connect your Quest Pro:** choose USB or wireless ADB and complete the connection steps in section 4.
2. **Install the PC runtime:** press **Install runtime** and wait for Activity to confirm completion. This prepares Qpro's private Python and downloads the PC parts needed for tracking. The Hub stays at your current place on the page while it runs. You do not need to install, repair, or remove another Python installation.
3. **Install your VRCFaceTracking module:** close VRCFaceTracking and wait for its module process to exit. Choose your **Streaming app**, press the matching **Install Virtual Desktop module** or **Install Steam Link module** button, wait for completion, then reopen VRCFaceTracking. Only the selected app's install button is available. Installing one Qpro module uninstalls the other Qpro source module. The module appears under its own Qpro name in VRCFaceTracking. Remove any separate Virtual Desktop or Steam Link source module through VRCFaceTracking before installing Qpro's module.

Module installation supports local app-folder junctions, directory symlinks and Windows Cloud Files. For OneDrive, choose **Always keep on this device** for the entire extracted Qpro folder. New module recovery copies are kept in `%LOCALAPPDATA%\QproFaceTracking\module-recovery`; Activity gives the exact folder after installation.

The **Qpro module** status checks the selected source's installed folder, module card and DLL against this app. **Installed · input unchecked** confirms the files match; it does not prove VRCFaceTracking has loaded them or is receiving expressions. **VRCFaceTracking: Running** only means that app is open. Check its preview for moving face expressions. If the Hub reports a different source, conflicting copy, damaged card or module update, open **Activity** for the reason, close VRCFaceTracking and install the matching module again. Restart VRCFaceTracking after a source change.

Optional compatibility details: Check headset compatibility reads the Quest Pro model, exact firmware build, tracking engine and gaze setup without changing headset tracking. **Show firmware details** explains which checks passed and what remains unverified. This diagnostic is useful when something fails; it is not an extra required step for ordinary face tracking or an active Magisk gaze module. Each selected Qpro feature still checks its requirements when it starts. A recognized Hub gaze engine is not proof of live input.

Optional independent gaze: if the Independent Eye Gaze Magisk module is enabled, **skip both Check gaze setup and Prepare gaze, and keep Independent Eye Gaze off in the Hub**. The module already handles independent gaze. If you choose the Hub's temporary independent gaze method instead, disable the Magisk module and reboot first, then use **Check gaze setup** followed by **Prepare gaze**. The popup identifies the detected gaze method and gives the next step. The Hub reads the stock eye model and tracking-engine details from that headset, then prepares a temporary model on the PC. You do not need to supply an engine file. Neither button is required when you only want tongue or pupil tracking. Prepare gaze again after a headset firmware update when using the Hub's gaze method.

The same card and result popup explain both recovery buttons: **Recover Qpro gaze** restores the previous eye-model state saved for a recorded Qpro session. **Reset legacy gaze** offers a confirmed choice of normal, nonexperimental gaze selection for an older session without a record. Neither button disables a Magisk module or uninstalls the PC module. The check reads headset settings; it does not test whether convergence works in your avatar.

The Hub checks the prepared model and headset build before restarting tracking. If the build is unsupported or changed, Activity explains why and the Hub leaves headset tracking alone. Keep the Hub's **Independent Eye Gaze** off and use ordinary eye tracking from your selected streaming app. Do not use another gaze method to bypass a failed compatibility check. Note your Quest's **exact firmware build** when asking for help. On a supported build, applying or restoring the Hub's temporary eye model restarts Meta trackingservice. The headset may briefly look frozen and lose positional tracking; this is an expected restart, not a headset crash. Wait for Activity to confirm that tracking has returned.

To remove Qpro's VRCFaceTracking add-on later, first press **Stop tracking**. Close VRCFaceTracking, wait for its module process to exit, open **Show module uninstall** on the **Install your VRCFaceTracking module** card and press **Uninstall Qpro module**. Confirm the prompt, wait for **Setup step complete**, then restart VRCFaceTracking with exactly one official module for your selected streaming app. The Hub removes verified Qpro modules and can restore a verified official Virtual Desktop module saved by an older installer. If no official module was restored, install the official Virtual Desktop or Steam Link module you need. Your recordings and models stay saved. If an older workaround copied a Qpro DLL into another module's folder, Qpro removes the verified copy and leaves that folder's other files and metadata alone; repair or remove its old card through VRCFaceTracking if needed. Removing the PC module does not change headset gaze settings or disable a Magisk module.

AMD GPU: After **Install runtime** succeeds, press **Install ROCm 10.1** if the Hub detects one of these discrete Radeon models on Windows 11. Windows 10 22H2 (build 19045 or later) users can explicitly enable **Try ROCm on Windows 10 (experimental)** before installing. AMD's supported Windows configuration is Windows 11; Windows 10 success depends on the card and driver and is not guaranteed. The same discrete GPU, training and inference checks still must pass. For a manual attempt, run `Install-QproRocm.ps1 -AllowExperimentalWindows10`. Older Windows versions and legacy ROCm 7.2.1 on Windows 10 are not admitted. If an older verified ROCm 7.2.1 environment is already present, the button says **Upgrade to ROCm 10.1** instead:

For the Windows 10 option, open **Show AMD compatibility details** on the **Optional AMD ROCm acceleration** card.

Series	Models mapped for ROCm 10.1
RX 6000	6950 XT, 6900 XT, 6800 XT, 6800; 6750 XT, 6700 XT, 6700; 6650 XT, 6600 XT, 6600
RX 7000	7900 XTX, 7900 XT, 7900 GRE; 7800 XT, 7700 XT, 7700; 7600 XT, 7600
RX 9000	9070 XT, 9070, 9070 GRE; 9060 XT, 9060
Radeon PRO	AI PRO R9700, PRO W7900, PRO W7900 Dual Slot

Qpro installs AMD TheRock ROCm 10.1 packages for the detected card's GPU target. These are AMD's stable packages, while their use in Qpro remains **experimental**. AMD's package index includes Windows device packages for the RX 6750/6700 family (gfx1031) and RX 6650/6600 family (gfx1032). Package availability does not guarantee driver compatibility on every card. Confirm the Hub prompt and allow time for a large download. AMD's ROCm 10.1 matrix validates Windows 11 25H2 with Adrenalin 26.10.41.05; other Windows 11 builds or drivers may fail the GPU checks. Qpro enables the new environment only after GPU training and model inference tests pass. Integrated graphics and unlisted Radeon models are not eligible. If setup fails, read the full **Activity** error.

New ROCm installations use a short, shared Qpro folder under `%LOCALAPPDATA%\QproFaceTracking\r`, with a separate folder for the card's GPU target. For example, an RX 7900 XT uses `10-gfx1100`. This avoids long library paths caused by extracting Qpro into a deeply nested folder. You normally do not need to change Windows registry settings. Qpro also keeps verified older `.venv-rocm-experimental` installations available. If this extracted copy has a **verified** `.venv-rocm` installation on a card in AMD's Windows ROCm 7.2.1 list, Qpro keeps it as a fallback when ROCm 10.1 is not ready. The Hub shows the version that was actually verified, including an existing ROCm 10.0 environment. An RX 7900 XTX was previously live-tested with ROCm 7.2.1 and 10.0. The new 10.1 recipe has passed offline checks but has not been installed or live-tested in this review. Other mapped cards still need physical checks; sharing a GPU target does not prove every model or driver works.

NVIDIA GPU or CPU: skip the AMD button. NVIDIA CUDA hardware has been live-tested and is functional. When tracking starts, **Activity** tells you whether lower-face inference and pupil processing are using AMD, NVIDIA, or CPU.

Optional: experimental hands and controllers (test build)

Hands + controllers uses the headset cameras to track your real fingers while you hold the physical controllers. Controller position, buttons and haptics still come from the controllers. This feature is separate from VRCFaceTracking's face module, is off by default, and currently requires **Virtual Desktop** on a rooted Quest Pro. Steam Link is not supported for this option. A previous live test showed finger movement while holding controllers, but also caused a substantial frame-rate drop. The current revision still needs a sustained live performance check.

> **Before enabling hands + controllers:** turn on Quest hand tracking and Singularity's simultaneous switch, connect through Virtual Desktop, and enable VRChat finger tracking. Installing Qpro's components does not enable these settings for you.

1. **Enable hand tracking on the Quest.** Open the headset's hand-tracking settings (under **Settings > Hands & Eyes** on the checked firmware) and turn on hand tracking. With controllers set down, check that the headset can see your empty hands. Keep the headset awake and grant Qpro's ADB shell root access when asked.
2. **Enable Singularity's simultaneous switch.** Open **Singularity > AIO Tweaks** and turn on **Simultaneous Hands & Controllers** (some versions label it **Simultaneous Hands Controllers**). Leave it on while using this feature. Turn off Singularity's separate **Frida Server** so it does not compete with Qpro's helper. The toggle is documented in Singularity's release notes; its location can vary between Singularity versions.

3. **Install Qpro's optional components once.** Install the normal **PC runtime** first, then close **SteamVR** and wait for it to exit. In **First-time setup > Optional hands and controllers**, open **Show hands setup and uninstall** and press **Install hand/controller components**. Setup downloads checksum-verified Frida 17.18.0 into Qpro's separate environment and registers Qpro's controller add-on. Repeat installation only when the Hub asks you to repair or update these components.
4. **Connect the VR session.** In the headset's Virtual Desktop **Streaming** tab, set **Forward tracking data to PC** and **Track controllers** on, and leave **Emulate Index controllers** off for the initial VRChat finger check. Accept the tracking-data permission prompt. Connect to the PC Streamer, then launch **SteamVR through Virtual Desktop**. A USB ADB connection alone does not start this VR stream. Launch the SteamVR version of VRChat and turn on **Main Menu > Controls > Avatars Use Finger Tracking**. This is VRChat's setting for displaying SteamVR skeletal input on the avatar. See the Virtual Desktop settings in section 3 and VRChat's finger-tracking instructions.
5. **Check this exact Virtual Desktop installation.** Select **Virtual Desktop** as the Hub's **Streaming app**. In **Live tracking > Hands and controllers**, press **Check hand/controller compatibility**. The admitted headset version is **Virtual Desktop 1.34.22.0**, with an exact checked PC Streamer driver. The original PC driver and the inspected **1.34.23** beta PC driver are admitted separately. Headset Virtual Desktop **1.34.23.0** still needs a checked hand layout. If your version or driver is unsupported, leave the option off and send its Activity details; reinstalling components cannot add a missing compatibility profile.
6. **Start and verify real fingers.** Turn on **Experimental hands + controllers**, leaving **Experimental Touch Pro thumb-rest input** off for this first check. Press **Start tracking**, hold the controllers with your hands visible to the headset cameras, and move individual fingers. Activity must progress beyond startup to `HANDS_READY`; that reports fresh optical data routed to at least one held controller. Check both hands in VRChat: fingers should follow your real fingers while controller movement and buttons still work. Ordinary fist, point or open-hand gestures from touching controller buttons do not establish that optical finger tracking works.
7. **Stop cleanly.** Press **Stop tracking** and wait for Activity to confirm controller cleanup before closing the Hub or restarting this feature. If cleanup is unconfirmed, keep the feature off, close SteamVR, restart headset Virtual Desktop and reopen SteamVR before checking again. If game FPS falls or SteamVR becomes unstable, stop tracking and keep this experimental option off.

Troubleshooting hands + controllers:

- **Only controller gestures move:** recheck Quest hand tracking, Singularity's simultaneous switch and VRChat's **Avatars Use Finger Tracking** setting. Keep fingers in camera view and read Activity for `HANDS_READY` or the routing failure. Being connected over ADB or seeing a helper start is not a live finger check.

- **Settings look off in Activity even though their menu switches are on:** preliminary hand-tracking preferences can be inactive before Virtual Desktop requests simultaneous mode. The current version warns on those known inactive values and then checks actual optical input. Missing or unreadable settings, simultaneous mode other than 1, an unsupported runtime, or missing live input still refuse startup. Check connection/root access and reconnect Virtual Desktop; do not edit the preferences by hand.

- **Unsupported version, driver or controller ABI:** keep Hands + controllers off and copy the full diagnostics. Face tracking can continue without this option. New Virtual Desktop versions need their own checked profile.

- **No fingers after startup or a disconnect:** keep the headset on, hold a controller, and keep the hand visible. Reconnect Virtual Desktop if its stream stopped, then use **Stop tracking** before retrying. The compatibility button checks installed versions and settings; it does not start tracking.

Optional thumb-rest input: Experimental Touch Pro thumb-rest input is separate from finger tracking. It adds SteamVR trackpad inputs from the controllers' thumb-rest sensors. Its first sensor profile is limited to firmware build **51503870024400340** and remains unvalidated on a live headset. Choose **Thumb-rest mode**: **Trackpad**, **Relative joystick**, **Swipe**, or **Desktop mouse**. Mouse mode explicitly moves the Windows pointer. Set the new trackpad bindings in SteamVR for your app; installing them does not automatically add VRChat actions.

Removing the add-on: close SteamVR, press **Uninstall controller add-on** in First-time setup, then reopen SteamVR to reload its normal controller profile. Uninstall keeps a recovery copy and removes the optional thumb-rest add-on; it does not remove the installed hands runtime components or turn off Singularity's switch.

See the experimental controller guide for compatibility and recovery details. Body-tracking interaction, sustained performance and newer runtime versions still need live checks.

6. Lower-face calibration: record and train tongue and cheeks

Train if you have no personal model or used a version before V2.0. You can import a working V2.0 or newer model into V3.0.0. The bundled models were trained on other wearers.

1. For Quick refinement or Focused training, choose the model you want to extend under **Live tracking > Lower-face tracking > Lower-face model** before training. Developer v8 is the default. Full dataset makes a new personal model instead.
2. Open **Personalize**. Start with **Quick refinement**, or press **Show focused and full captures** for the other capture modes below.
3. Follow each pose card, hold the pose steady, and press **SPACE** once per still. Save at least the on-screen minimum for every card before pressing **ENTER** to continue. Include the side and **diagonal** positions; missing a pose can leave a tracking gap there.
4. When recording finishes, choose it in the training list and press that mode's training button. Wait for Activity to say training finished. Select the resulting model when you restart tracking in section 7.

- **Quick refinement - 15-30 min:** press **1. Record refinement**, then **2. Train personalized copy**. Its 37 cards refine the selected tongue model and include 21 cheek camera cards for left, right and two-cheek puff strengths.

- **Focused diagonals + facial hair - 15-30 min:** press **1. Record focused dataset**, then **2. Train focused copy**. Its 22 cards add diagonal positions and matched hidden/visible poses for facial-hair shadows.

- **Full dataset - 60-120 min:** press **1. Record full dataset**, then **2. Train new personal model**. Its 79 cards record the full tongue curriculum followed by 21 cheek camera cards, including relaxed cheeks, different puff strengths and non-puff expressions. Take breaks as needed.

If you have facial hair or wear a bandage near the mouth, keep it as you normally wear it during tracking. For every visible pose, check that the tongue can actually be seen in the two camera panels. Hold one pose per still; vary your jaw or headset position slightly between stills. Record hidden poses too, so beard shadows and smiles are not mistaken for tongue movement. Hair that completely covers the tongue cannot be removed by training or contrast processing. Tongue cards need VRCFaceTracking to keep sending fresh face-tracking data; the expression values can stay the same while you hold a pose. Cheek cards use the pose shown on screen as their training label, so a weak native cheek signal does not block them. Follow the requested strengths approximately; these are animation targets, not measured air pressure.

Using camera cheek puff

Developer tongue + cheeks is an optional experimental copy of developer v8 with camera cheek outputs trained on one wearer. The original developer v8 remains the default, and its tongue weights are unchanged in this copy. Another wearer or headset fit may need personal calibration.

Quick refinement and Full dataset make a combined tongue-and-cheek model when you complete the cheek cards. The older **Focused diagonals + facial hair** workflow still trains tongue outputs only. Old tongue-only captures and imported models remain usable; importing them does not add camera cheek outputs.

To try a combined model, open **Live tracking > Lower-face tracking**, choose it under **Lower-face model** and turn on **Camera cheek puff (experimental)**. Selecting the model alone does not enable camera cheeks. The message below that switch explains the selected cheek source. Stop and restart tracking to apply the change; **Activity** reports which camera outputs were requested and the loaded model paths.

The model predicts separate left and right strengths between 0 and 1 from the mouth cameras. It can run with tongue tracking off. Camera cheek output does not use the short native cheek calibration or its response style. Turning camera cheek output off or losing its live feed returns to your selected cheek style while Qpro tracking runs. **Stop tracking** or closing the Hub restores the streaming app's original cheek values.

The **Experimental tongue + cheeks model** card on **Personalize** lets you add just the 21 cheek cards to a new copy of an existing tongue model. Press **Show advanced cheek training**, choose **Parent tongue model**, then press **1. Record cheek camera poses**, then choose the completed recording and press **2. Train tongue + cheeks copy**. This freezes the tongue model while learning its cheek outputs; the original model stays saved. Test the new copy with fresh poses before relying on it. Camera cheek tracking remains experimental and may need a personal recording for another face or headset fit.

Trying Mustachio

Mustachio is highly experimental and optional. It extends developer v8 with a focused capture from one bearded and moustached wearer. Independent clean-shaven testing and broader facial-hair testing are still pending, so it may perform worse for some people. Developer v8 remains available and is the default.

To try it, choose **Mustachio** under **Live tracking > Lower-face tracking > Lower-face model**, then restart tracking. Its contrast processing is selected automatically by the model; it does not invent hidden tongue pixels or remove hair. Quick refinement and Focused training can extend this selected model. Those copies retain the **highly experimental** label, including after renaming, export, and import. Choosing Full dataset starts a new personal model.

After training, **Activity** lists the weakest held-out pose cards and diagonal corners. `missed` counts visible-tongue frames that the selected visibility gate marked hidden; `fnr` is that count divided by visible frames. These are checks on held-out frames from the same recording, so try the model live as well.

The **Model manager** can export a model as a backup. Import working V2.0 or newer exports into V3.0.0; for pre-V2.0 exports, make a new capture and train again. Camera recordings are personal data, so share them only if you want to.

To remove a recording you no longer need, stay on **Personalize**. Under its matching capture card, open **Show saved recordings**, choose it from **Saved recordings (including trained)** and press **Delete selected dataset...**. Read the confirmation before choosing **Yes**: this permanently removes the recording, its labels and session details, and its prepared training cache from this extracted copy. A tongue model already trained from that recording remains available in **Model manager**. The training dropdown above only lists datasets still waiting to be trained.

7. Start tracking

1. Start your selected **Virtual Desktop** or **Steam Link** app on the Quest, then SteamVR and VRCFaceTracking on the PC. Steam Link users should confirm its OSC and eye/face sharing settings above.
2. In the Hub, open **Live tracking**. The **Eyes** section contains independent gaze, pupil dilation and eyebrow movement. Independent Eye Gaze starts off. **If you installed Singularity's Independent Eye Gaze Magisk module, leave the Hub's Independent Eye Gaze option off.** In **Lower-face tracking**, choose the **new model you just trained** under **Lower-face model** and enable the camera features you want. Its cheek controls are in the same section. Optional **Hands and controllers** controls are at the bottom. If tracking is already running, press **Stop tracking** first so the new model loads when tracking restarts.
3. Press **Start tracking**. Tracking camera previews default off on a fresh setup; saved preview choices are kept. To show them, open **Your session > Show connection and camera settings** and enable **Preview tracking cameras** before starting. Guided training capture still opens its pose window. Open **Activity** if you want to see whether the camera connected and which device is running the tongue model.
4. When you finish, press **Stop tracking**. Wait until Activity confirms that the live processes have stopped. If you used the Hub's gaze method, also wait for its gaze recovery result: it restores the headset eye-model state recorded before that Qpro session.

Closing the Hub stops tracking cleanly, then stops the configured PC ADB server. Other Android tools using the same ADB server will disconnect and may restart it. Finish setup, capture, training, or eye-model recovery before closing.

Stopping is different from uninstalling. Tongue, camera cheek and pupil output stop when Qpro tracking stops. The PC tongue model is not installed on the headset. **Individual cheek puff** and **Individual cheek suck** apply only while Qpro tracking runs; **Stop tracking** or closing the Hub restores the streaming app's original cheek values. Your selected styles and personal calibration stay saved for the next session. You can start a session with just these cheek adjustments selected; no camera or Python process is needed. **Adjust eyebrow movement** and the module's built-in smirk adjustment remain active through the installed module. Turn the eyebrow switch off for native eyebrows, or uninstall the Qpro module for the official module's complete mapping. Magisk gaze modules remain active until you disable them in Magisk and reboot.

Returning to ordinary tracking

Use **Check gaze setup** in **First-time setup** for optional troubleshooting, or before choosing the Hub's temporary gaze method. It reads the headset's current eye-model selection and active gaze methods without changing them. It is not required when an Independent Eye Gaze Magisk module is enabled; leave the Hub's **Independent Eye Gaze** off and skip both gaze setup buttons.

- **An interrupted Qpro gaze session:** After stopping Qpro tracking, use **Recover Qpro gaze** in **First-time setup**. It restores a verified session's recorded previous state. A result saying no Qpro session remains does not mean every headset modification was removed.

- **An older build left experimental gaze enabled without a recovery record:** Use **Reset legacy gaze** in **First-time setup** only when you want the normal, nonexperimental headset gaze selection. Read and confirm the prompt. This cannot recover an unknown previous selection; it explicitly chooses the normal selection. It refuses active independent-gaze Magisk modules or an unverified overlay on the eye model or its tracking engine. It does not uninstall modules or overwrite headset firmware. If a gaze Magisk module is active, disable it and reboot before retrying.

- **Compare the streaming app's original face tracking:** Stop Qpro tracking, close VRCFaceTracking and wait for its module process to exit. Use **Uninstall Qpro module**, then reopen VRCFaceTracking with exactly one official module for Virtual Desktop or Steam Link. To compare ordinary headset eye tracking too, disable any independent-gaze Magisk module and reboot the headset.

If either recovery action refuses to proceed, keep its **Activity** error and ask for help with the exact firmware build. Recovery does not add support for an unsupported eye-tracking engine.

On **Live tracking**, open **Your session > Show connection and camera settings** for **Preview tracking cameras** and **Camera FPS cap**. Open **Lower-face tracking > Show camera settings** for **Motion smoothing** and **Tongue visibility**. Hiding the camera windows does not turn tracking off. Stop tracking before changing these settings, then start again. In **Eyes > Show eye settings and guidance**, **Pupil response** makes avatar pupil changes stronger or weaker; start near the default and adjust slowly. Pupil values are estimates for avatar animation, **not** measured eye health data. Your VRChat avatar must support pupil animation for changes to appear.

Pupil processing and performance

Pupil tracking automatically uses a verified NVIDIA CUDA or AMD ROCm runtime when one is available. Filtering both eye images is batched on the GPU; contour fitting, quality checks and smoothing still use the CPU. If GPU processing is unavailable or fails, Qpro reports the fallback in **Activity** and continues on CPU. The **Pupil tracking** status under **Your session > Show connection and camera settings** shows whether the work is running on AMD, NVIDIA or CPU.

Qpro processes the latest eye pair instead of queuing old frames. Activity reports processing time, frame age and skipped work so a slow PC is easier to diagnose. Camera previews encode only the streams somebody is viewing. Close unused browser previews if performance is poor; GPU support cannot guarantee a particular frame rate on every PC.

Choose your cheek puff response

On **Live tracking**, find the cheek controls in **Lower-face tracking**. **Individual cheek puff** starts on with **1/0** selected. These controls adjust cheek puff from your streaming app only while Companion tracking runs, when **Camera cheek puff (experimental)** is off or its output stops. Choose a **Cheek puff style**:

- **Calibrated:** a smooth strength from relaxed to full puff, including values between 0 and 1. It uses your personal calibration for the selected streaming app, or the bundled developer cheek baseline until you make one. Separate developer baselines were measured on one Quest Pro through Virtual Desktop and Steam Link. They are a starting point; calibrate for your own face if the response is too weak or too strong.

- **1/0:** a clear one-cheek puff becomes full strength on that side and zero on the other. Puffing both cheeks still moves both.

- **Balanced:** gentler separation that keeps changes in cheek strength without using a calibration profile.

With camera cheek output off, turn **Individual cheek puff** off to use the original cheek values from Virtual Desktop or Steam Link. Press **Start tracking** in the Companion to apply your selected cheek style. **Stop tracking**, closing

the Companion, or a Companion crash restores native cheeks; your selections and calibration stay saved for the next start. If the VRCFaceTracking preview separates the cheeks but the avatar does not, check the avatar's parameters and blendshapes.

Calibrate cheek puff for your face

1. Keep the headset on and your chosen streaming app connected. Its matching Qpro module must be installed and tracking in VRCFaceTracking. **Qpro camera tracking does not need to be running.**
2. On **Live tracking > Lower-face tracking**, press **Calibrate cheek puff** beside **Cheek puff style**. Confirm that the window says **Live cheek feed is ready**.
3. Relax both cheeks, press **Capture relaxed cheeks**, and hold still for three seconds.
4. Puff only your own left cheek, keeping the right relaxed. Press **Capture left cheek** and hold for three seconds. Relax briefly afterward.
5. Puff only your own right cheek, keeping the left relaxed. Press **Capture right cheek** and hold for three seconds.

The window saves your profile after all three poses pass, then selects **Calibrated** and turns **Individual cheek puff** on. Keep your jaw comfortable and avoid smiling during the holds. This three-pose calibration uses the module's **Balanced** cheek strengths as its input. If a pose is too weak, unstable, or shows both cheeks together, follow the message and repeat that step. A paused or mismatched feed must be fixed before you continue. Closing the window before completion keeps any existing profile.

Updating from the four-pose test: Close VRCFaceTracking, install the matching Qpro module from this build, then reopen it. Earlier three-pose profiles still work. Four-pose profiles stay saved but are not used by this version; **Calibrated** uses the bundled developer baseline until you complete a new three-pose calibration. You can wait until you have the headset available.

Virtual Desktop and Steam Link have separate personal profiles. Calibrate each streaming app you use; switching back uses that app's saved profile. This is a short cheek calibration, so you do not need to record or retrain your tongue model.

Individual cheek suck uses the separate left and right cheek-suck signals from Virtual Desktop or Steam Link. It starts on with **Strong individual (1/0)**, which emphasizes the stronger side while leaving a deliberate two-cheek suck on both sides. Choose **Balanced** for a softer effect, or turn it off to pass through the streaming app's original values. It does not use negative cheek-puff values.

In **Live tracking > Eyes**, **Adjust eyebrow movement** starts off, so brows use the selected source's original values. Turn it on, open **Show eye settings and guidance** and adjust **Eyebrow sensitivity** to amplify or soften the existing left and right inner raise, outer raise, and lower/pinch expressions. This cannot add movements the headset does not detect. The avatar needs matching brow parameters and blendshapes to show that detail. If the expressions move in VRCFaceTracking's preview but the avatar shows only a single brow motion, check the avatar's face tracking setup.

Smirk handling is built into both Qpro modules. A clear one-sided smile emphasizes its leading corner and suppresses the weaker side; an even smile still moves both sides. There is no separate smirk toggle in the Hub.

For tongue motion, adjust **Motion smoothing** to soften extension and retraction. The visibility hold reduces quick dropouts after a visible pose, but it cannot make a completely obscured tongue visible. Use **Focused diagonals + facial hair** to record positions that still need better coverage.

Common problems

If Independent Eye Gaze fails to start or stops unexpectedly, the Hub turns it off automatically and remembers that choice. Tongue and pupil tracking continue if selected. Check **Activity** for the recovery result before trying gaze again; select **Independent Eye Gaze** manually to retry.

What you see	What to try
"Root access unavailable"	Keep the Quest awake. In Magisk, allow **Shell / ADB Shell**, then refresh the connection.

What you see	What to try
USB Quest is offline or the connection stalls	Keep the headset awake. Stop Qpro tracking and training, then use **Reconnect USB** in **First-time setup**. If USB debugging is unauthorized, allow it inside the headset and choose **Always allow from this computer**. Check the cable and Windows USB driver if the device is still missing.
A different USB Quest Pro is connected	Stop Qpro tracking and training, then use **Forget USB headset**. Connect the intended Quest Pro and press **Reconnect USB** to verify and remember it; temporarily disconnect other USB Android devices for its first check.
VRCFaceTracking is running but the Qpro module needs attention	Open **Activity** for the module check's reason. Close VRCFaceTracking and wait for its module process to exit, then install the matching Qpro module. The running app status does not verify the installed module or live face input.
Quest connected pop-up, followed by a pairing error	You are already connected. Skip **Pair and connect** and start tracking.
Quest connection failed pop-up	Check the Quest's current Wi-Fi IP and port, wake the headset, and read **Activity** for the specific error.
`protocol fault` during pairing	Use a fresh six-digit code and the **temporary pairing port** shown on the Quest, not the usual `:5555` port.
Wireless stopped working after a reboot	Check the Quest's current Wi-Fi address. Wireless ADB may need to be enabled again.
AMD installer says PC runtime is missing	Finish **Install runtime** first, then return to **Install ROCm 10.1**.
ROCm setup reports a missing `torchgen` or a failed Python import	Use **Verify / repair ROCm 10.1** or **Repair ROCm 10.1** in First-time setup. Qpro checks and repairs its separate AMD environment before testing the GPU. Keep the full **Activity** error if repair fails; an import error alone does not mean your card is unsupported.
Install runtime stays on "Starting PC runtime setup"	Keep the Hub open and read **Activity**. It now shows the PowerShell launch, current setup phase, and a still-running message every 15 seconds when output stops. If PowerShell never prints its first script message, check Windows Security for a blocked `powershell.exe`, then share the Activity lines, including the PowerShell PID.
Install runtime ends with code 1	Read the explanation and first error in **Activity**. A missing native DLL, `VCRUNTIME`, `WinError 126` or `WinError 1114` calls for installing or repairing Microsoft's latest x64 Visual C++ Redistributable, then retrying. `ModuleNotFoundError` means a Python package or app file is missing; follow Activity's runtime repair or complete-ZIP instructions instead. Code 1 has other causes too, so share the full error if it repeats. Do not delete the whole `%LOCALAPPDATA%\QproFaceTracking` folder just because the exit code is 1.
Steam Link source selected but no face or training labels arrive	In Steam Link **Advanced Settings**, turn on **OSC**, **Share eye tracking data**, and **Share face tracking data**; set **OSC Output Port** to **9015**. Close VRCFaceTracking and wait for its module process to exit, then press **Install Steam Link module** in the Hub. Reopen VRCFaceTracking. Remove other Steam Link VRCFaceTracking modules that would use the same port.
Launcher says the tracking source does not match	Close VRCFaceTracking and wait for its module process to exit. Choose the intended **Streaming app** on **First-time setup**, install its Qpro module, reopen VRCFaceTracking, then retry.
Calibrate cheek puff is waiting for live values	Keep the headset on and the selected streaming app connected. Start tracking in VRCFaceTracking with the matching Qpro module from this build. If the window reports the other streaming app, select the correct **Streaming app** in the Hub or install its matching module with VRCFaceTracking closed. Qpro camera tracking is not required.

What you see	What to try
A combined model moves both cheeks but separate puffs do not respond	In **Lower-face tracking**, check the cheek-source message and enable **Camera cheek puff (experimental)** if you want the camera model. Restart tracking and check **Activity** for camera cheek output ON and the selected direction model. If it is already ON, keep the log: an unfamiliar face or headset fit can still need personal cheek capture. Native cheek calibration does not train the camera model.
AMD Ryzen integrated graphics appears during ROCm setup	Qpro checks for one of the exact discrete Radeon models in the ROCm 10.1 table above. An integrated GPU alone is not enough. Check that Windows and the AMD driver can see the discrete card; otherwise use **Install runtime** for NVIDIA CUDA or CPU.
Independent gaze or convergence fails but ordinary face tracking works	If you use the Magisk gaze method, keep Hub gaze off and check that module's firmware requirements; the Hub's gaze setup is not required. For the Hub's temporary method, use **Check gaze setup** and read Activity for the exact firmware build and next step. Keep Hub gaze off if its checks fail. Use only one gaze method.
Tongue disappears in some positions	Check the selected model and tongue output state in **Activity**. A stale result means the frame arrived too late; check the selected GPU runtime and reduce other camera work. Hidden output means the visibility gate did not accept that pose. If fresh poses still fail, keep the log and record those positions for refinement when possible.
Training says the prepared tongue cache uses an older mapping	Retry training in the Hub to rebuild prepared data from the saved capture. Keep the original recordings. If an older recording has an ambiguous streaming source, copy its diagnostic and identify whether it was recorded in Virtual Desktop or Steam Link before preparing it again. Existing trained models remain usable.
Facial hair or a bandage makes tongue training unreliable	Record a personal dataset with the same face appearance you use for tracking. Capture hidden and visible tongue poses, then check the trained model in Live tracking. If native visibility flickers while the cameras still see your tongue, try **Camera only** under **Show camera settings > Tongue visibility**. This may help but is not guaranteed for every face or headset fit.
Pupils seem too jumpy	Stop tracking, open **Eyes > Show eye settings and guidance**, lower **Pupil response**, then start again. Hold your gaze steady while tracking warms up.

If Windows will not let you delete an older extracted Qpro folder, close its Hub and camera preview first. If the folder is still in use, open **Task Manager > Details**, end a leftover `adb.exe` process, then try again. Ending `adb.exe` temporarily disconnects any other Android tools using that ADB server.

If you need help, open **Activity** and press **Copy diagnostics** or **Save log...**. Include the error, your Quest's exact firmware build, whether you used USB or wireless, and which release ZIP you downloaded. You can ask in the community Discord. The server is not owned by Fwooffy.

The Steam Link source follows the OSC mapping documented by danwillm and the LinkFT project. Qpro does not require either third-party module alongside its own module.

Tested PC example

The owner reported good performance with few issues on Windows 11 Pro, an Intel Core i7-13700KF, AMD Radeon RX 7900 XTX, and 32 GB RAM. This is an experience report, not a benchmark or a guarantee for other PCs.